

grplot: A Python Library for Lazy Statistical Data Visualization

Ghiffary Rifqialdi ^{1,2,3¶}

¹ University of Hamburg, Hamburg, Germany ² University of L'Aquila, L'Aquila, Italy ³ Institut Teknologi Bandung, Bandung, Indonesia ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Abhishek Tiwari](#) 

Reviewers:

- [@rahuldevikar](#)
- [@kevinrue](#)
- [@causalfragility-lab](#)

Submitted: 25 February 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

grplot is an open-source Python library that reduces multi-step statistical plotting workflows to a single high-level function call. Built on top of Matplotlib ([Hunter, 2007](#)), NumPy ([Harris et al., 2020](#)), and Pandas ([McKinney, 2010](#)), and bundled with a vendored Seaborn fork (grplot_seaborn) that internally uses SciPy ([Virtanen et al., 2020](#)), it exposes a unified plot2d API that automatically handles subplot layout, axis labeling, legends, statistical annotations, tick-label number formatting (thousand separators, currency symbols, and magnitude abbreviations), and figure export to PNG, PDF, SVG, and EPS. Users specify what to plot via a plot-type string or a dictionary mapping panel positions to plot types; the library applies sensible defaults while accepting more than 100 parameters for explicit overrides at global, per-axis, or per-element granularity. As of version 1.0.6, grplot requires Python 3.10+ and is available on PyPI (v1.0.6) and conda-forge (currently distributed as v1.0.4; the conda-forge feedstock lags PyPI releases) ([Rifqialdi, 2026](#)). Full documentation is available at grplot.readthedocs.io.

Statement of Need

Existing Python and R visualization libraries—Matplotlib ([Hunter, 2007](#)), Seaborn ([Waskom, 2021](#)), plotnine ([Kibirige & others, 2022](#)), and ggplot2 ([Wickham, 2016](#))—already produce publication-quality figures, and their grammar-of-graphics or axis-level APIs give users fine-grained, composable control over ticks, legends, and output. grplot's contribution is not a replacement for these tools, but two additions that practitioners otherwise implement themselves, project after project:

1. **An inset annotation and unit-formatting system.** grplot computes and renders descriptive-statistic blocks (quantiles, whiskers, confidence intervals), bar/point value labels, and locale- and magnitude-aware tick formatting (thousand separators, currency symbols, K/M/B/T abbreviation) directly on the axis, driven by declarative arguments rather than manual `ax.text/FuncFormatter` calls. This is the part of grplot that has no direct equivalent in Matplotlib, Seaborn, plotnine, or Altair, and is intended to compose with those tools' own axes rather than supersede them—grplot reuses standard Axes objects throughout.
2. **Two domain-specific analytic utilities**—cohort retention analysis and rank-order/gain/KS/lift tables—that practitioners in modeling and customer-analytics workflows commonly need to reimplement from scratch, with no equivalent single-call implementation in the packages above.

The multi-panel plot2d convenience call that composes these features into one function is a secondary, complementary layer: it is most useful for quickly assembling *heterogeneous* dashboards (e.g., a histogram, a Pareto chart, and an annotated box plot in one figure), where

each panel type would otherwise require bespoke annotation code (see Software Design for a quantified comparison). For homogeneous grids of a single chart type, a short loop over the underlying plotting library is often just as concise, and `grplot` does not claim an advantage in that case.

State of the Field

Several Python libraries address statistical data visualization, each with a different scope. Matplotlib (Hunter, 2007) provides a complete 2-D graphics environment with full control over every element, but requires verbose, procedural code for even routine plots. Seaborn (Waskom, 2021) raises the abstraction level for common statistical charts while remaining tightly coupled to Matplotlib's axis-management model. Altair (VanderPlas et al., 2018) and plotnine (Kibirige & others, 2022) implement declarative grammars of graphics (Wickham, 2016) that are elegant for exploratory work but do not natively support multi-panel layout, number formatting, or annotation in a single call. Plotly (Plotly Technologies Inc., 2015) excels at interactive web-based visualization but is not oriented toward static, publication-ready figures.

None of the tools above ships an inset statistical-annotation system (quantile/whisker/CI blocks, value labels, magnitude- and locale-aware tick formatting) as a declarative, reusable layer, nor the cohort and rank_order domain-specific analytics `grplot` provides; practitioners currently reimplement these per project using each library's lower-level Axes/text/Formatter primitives. `grplot` was built to package this recurring annotation and analytics work as a standalone contribution, implemented on top of—rather than in place of—Matplotlib and a vendored Seaborn fork. Its optional `plot2d` multi-panel convenience layer is a secondary, narrower contribution: useful for quickly assembling heterogeneous dashboards combining several chart families, at the cost of the modularity and composability that grammar-of-graphics libraries like `ggplot2` and `plotnine` deliberately optimize for instead (see Software Design).

Software Design

Hierarchical Argument System

The central design challenge was exposing a large surface area of configuration (20 chart types, multi-panel grids, per-axis formatting, per-element overrides) through a single function without requiring users to understand its full breadth for routine use. `grplot` resolves this with a four-level hierarchy of argument granularity:

- **Ordinary:** applied to the entire figure (e.g., `df`, `figsize`, `Nx`).
- **Axes:** scoped to a specific subplot by 1-based index `"[i]"` (1-D) or `"[row,col]"` (2-D grid), e.g., `plot`, `filter`, `title`.
- **Axes-plot:** scoped to a specific chart layer within a subplot (e.g., `hue={"[1,2]": {"scatterplot": "species"}}`).
- **Axes-axislabel:** scoped to a specific axis label within a subplot (e.g., `statdesc={"[1,1]": {"total_bill": "general"}}`).

Almost all axes-axislabel arguments apply to both axes by default; prefixing with `x` or `y` targets a single axis (e.g., `xlim`, `yrot`). This design deliberately trades away the lowest-level Matplotlib configurability in exchange for allowing a complete, multi-panel, annotated figure to be expressed in one call with consistent, predictable defaults. Full parameter documentation is available in the [online documentation](#).

Design Trade-off Relative to Grammar-of-Graphics Libraries

This single-call design is a deliberate departure from the grammar-of-graphics philosophy of `ggplot2` and `plotnine`, which favor small, composable, independently testable functions over configuration-heavy entry points. That approach optimizes for modularity, maintainability, and code reuse across distinct plots; `grplot`'s approach instead optimizes for a narrower target workflow—producing many *heterogeneous*, consistently annotated panels in a single notebook or report—where repeating boilerplate across chart families is the more common failure mode. To quantify where this trade-off actually pays off, we compared line counts for two equivalent dashboards implemented with `grplot` and with vanilla `Matplotlib/Seaborn`:

Dashboard	<code>grplot</code>	<code>Matplotlib/Seaborn</code>	Ratio
6 heterogeneous panels (histogram+KDE+stats, ECDF, treemap, pie, Pareto, box+strip+CI)	32 lines	101 lines	3.2×
9 homogeneous panels (boxplot only, repeated across a metric/cluster grid)	37 lines	34 lines	~1.0×

The advantage is concentrated in the first case: mixing several chart families that each require their own statistical-annotation code (quantile lines, a twin-axis cumulative curve, confidence intervals) is exactly where manual `Matplotlib/Seaborn` code grows and becomes inconsistent across panels as the number of analyses increases. For a grid of one repeated chart type, a short loop over `Matplotlib/Seaborn` is equally concise, and `grplot` claims no advantage there. Because `plot2d` returns the underlying `Matplotlib` Axes array, users retain full access to the layered `Matplotlib` API for any per-panel customization beyond what `grplot`'s arguments expose, rather than losing that granularity.

Supported Chart Types

`grplot` wraps 20 chart types across four families:

Family	Chart types
Relational	<code>scatterplot</code> , <code>lineplot</code>
Distribution	<code>histplot</code> , <code>kdeplot</code> , <code>ecdfplot</code> , <code>rugplot</code> , <code>pieplot</code> , <code>treemapsplot</code> , <code>packedbubblesplot</code>
Categorical	<code>stripplot</code> , <code>swarmplot</code> , <code>boxplot</code> , <code>violinplot</code> , <code>boxenplot</code> , <code>pointplot</code> , <code>barplot</code> , <code>countplot</code> , <code>paretoplot</code>
Regression	<code>regplot</code> , <code>residplot</code>

`treemapsplot` bundles an inline implementation of the squarified treemap layout algorithm described by Laserson (2013), requiring no external squarify dependency. Any two chart types may be overlaid on the same axis using `+` notation (e.g., `"histplot+kdeplot"`). Five composites carry pre-tuned default values: `boxplot+stripplot`, `violinplot+stripplot`, `boxplot+swarmplot`, `violinplot+swarmplot`, and `stripplot+pointplot`. Multiple panels can be composed into grid dashboards using `Nx` (columns) and `Ny` (rows), as illustrated in Figure 1.

Vendored Seaborn Fork

`grplot` ships a vendored fork of `Seaborn` (`grplot_seaborn`) to decouple production software that embeds `grplot` from upstream `Seaborn` breaking changes. This is a deliberate stability trade-off: users gain version-independence at the cost of not automatically inheriting `Seaborn` upstream improvements. The fork is kept up to date with stable `Seaborn` releases as part of `grplot` maintenance.

117 Analytic Utilities

118 `grplot.analytic.cohort` produces a cohort retention heatmap from a customer
119 transaction table, computing monthly retention rates in a single call (Figure 2). When
120 `display_summary=True`, the underlying cohort pivot table is also printed to the notebook
121 output for inspection.

122 `grplot.analytic.rank_order` produces a rank-order table with cumulative gain, KS statistic,
123 and lift per decile from predicted probabilities and true binary labels, supporting multi-class
124 outputs via a class selector. These utilities follow standard industry conventions and remove a
125 common source of bespoke, error-prone reimplementations in data science notebooks.

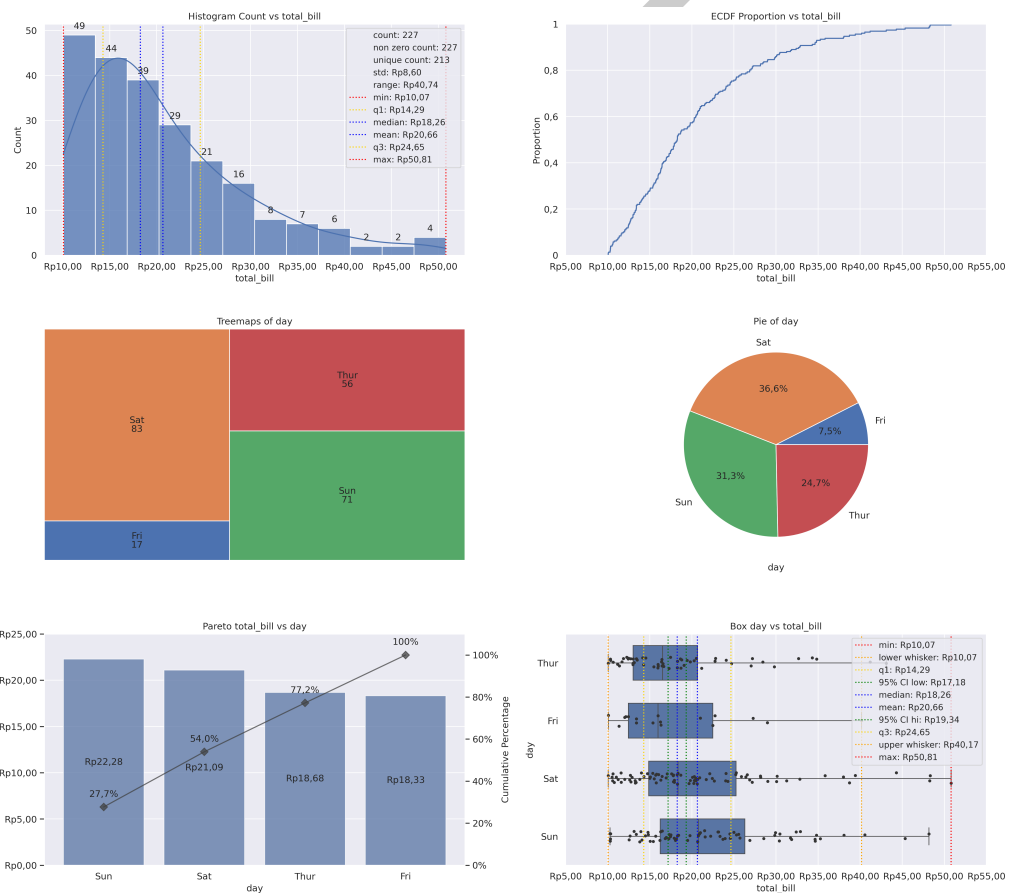


Figure 1: Six-panel 2x3 grid dashboard—histogram, ECDF, treemap, pie, Pareto, and box+strip composite—generated with a single `plot2d` call and a per-panel row filter applied to the Seaborn tips dataset. Tick formatting (`Rp(_)`), inset statistical annotation blocks, and bar-top value labels are all configured through `plot2d` parameters without any post-processing.



Figure 2: Monthly cohort retention heatmap produced by `grplot.analytic.cohort` from a retail transaction dataset. Rows represent cohort groups (signup month); columns represent cohort periods (months since first purchase); cell values show the percentage of customers active in each subsequent month.

Research Impact Statement

`grplot` reduces the time and code required to produce annotated, publication-ready figures in Python. By consolidating multi-step Matplotlib/Seaborn workflows into a single `plot2d` call with a hierarchical parameter system, it lowers the barrier to exploring and communicating data for data analysts, researchers, and data scientists who may not have deep expertise in lower-level graphics APIs. The bundled analytic utilities (`cohort` and `rank_order`) further accelerate common modeling-evaluation and customer-analysis workflows that practitioners would otherwise rebuild from scratch. `grplot` supports reproducibility by making figure-generation code concise, readable, and easy to version-control, and it integrates naturally into Jupyter notebook environments widely used in data science research.

An interactive [Colab documentation notebook](#) allows practitioners to run all examples in a zero-install environment.

AI Usage Disclosure

An AI assistant was used solely to assist with brainstorming and idea development during the writing of this paper. The tool was not used in software creation, code generation, or documentation writing. All technical content, design decisions, and architectural choices are the work of the author alone. All AI-assisted ideation was reviewed, evaluated, and validated by the author before inclusion.

Acknowledgements

The author thanks the maintainers of Matplotlib (Hunter, 2007), Seaborn (Waskom, 2021), NumPy (Harris et al., 2020), Pandas (McKinney, 2010), SciPy (Virtanen et al., 2020), and IPython (Pérez & Granger, 2007) for providing the foundational infrastructure on which grplot is built. The author also thanks renardelyon for early contributions to unit testing in the initial development phase. No financial support was received for this work. The author declares no conflict of interest.

References

- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Kibirige, H., & others. (2022). Plotnine: A grammar of graphics for Python. <https://doi.org/10.5281/zenodo.1325308>
- Laserson, U. (2013). Squarify: Pure Python implementation of the squarify treemap layout algorithm. <https://github.com/laserson/squarify>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Pérez, F., & Granger, B. E. (2007). IPython: A system for interactive scientific computing. *Computing in Science & Engineering*, 9(3), 21–29. <https://doi.org/10.1109/MCSE.2007.53>
- Plotly Technologies Inc. (2015). Collaborative data science. <https://plot.ly>
- Rifqialdi, G. (2026). Grplot: A python library for lazy statistical data visualization (Version 1.0.6). <https://github.com/ghiffaryr/grplot>
- VanderPlas, J., Granger, B. E., Heer, J., Moritz, D., Wongsuphasawat, K., Satyanarayan, A., Lees, E., Timofeev, I., Welsh, B., & Sievert, S. (2018). Altair: Interactive statistical visualizations for Python. *Journal of Open Source Software*, 3(32), 1057. <https://doi.org/10.21105/joss.01057>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>
- Wickham, H. (2016). ggplot2: Elegant graphics for data analysis. Springer. <https://doi.org/10.1007/978-3-319-24277-4>